

Calculator with Exceptions

Luca Roversi

December 10, 2025

Learning outcomes

- Have a look at subtraction on $\mathbf{BN} \rightarrow \mathbf{BN} \rightarrow \mathbf{BN}$
- Extend expressions with quotient/remainder

Expressions extended with subtraction and quo/rem

- `Numbers.Positive.OperationsBinary.hs` contains:
 - `Mask` with `IsNeg`, `IsNul`, `IsPos`
 - `sub :: Positive -> Positive -> Mask`
- `Numbers.Binary.Applicative.OperationsBinary.hs` contains:
 - `sub :: BN -> BN -> BN`
- `Numbers.Binary.Applicative.IsNumericalSystem.hs` contains:

```
instance Num BN where ...  
  (-) :: BN -> BN -> BN  
  (-) = sub
```
- `Arithmetic.Expressions.Sub.hs`
- `Arithmetic.Interpreters.Natural.Sub.Eval.hs`
- `Arithmetic.Expressions.QuoRem.hs`

A Question (LAB)

How to extend the interpreter built so far:

- to correctly compute the value of expressions with quotient/reminder
- to rise "Div by 0" exception when necessary?

A naive solution.

- `Arithmetic.Interpreters.Monadic.QuoRem.EvalNaive`

Alternative.

- “Delegate” to a dedicated composition the role to discriminate between `pure` and “computational effects” (like rising errors)